

EECE 4830-5830 Network Design, Dr. Vinod Vokkarane

Programming Project Phase 1

File Transfer between a UDP Client and a UDP Server

Project description: The TCP/IP stack has five layers, namely application, transport, network, link, and physical. In Phase 1(a) of the project, each student must individually implement the standard user datagram protocol (UDP) sockets. The intention is to transfer a message (Say “HELLO”) from the UDP client to the UDP server and then **ECHO** the message back from the UDP server to the UDP client. Note that the client and server process can reside on the same host but have to use different port numbers. **Make sure that your program can send and receive messages in both directions.**

Phase 1(b): In the second part of this phase, the intention is to transfer a file (say BMP) between a UDP client process and a UDP server process. Each student must implement the RDT 1.0 protocol described in Section 3.4.1 of the course textbook.

The end-to-end system architecture is given below:

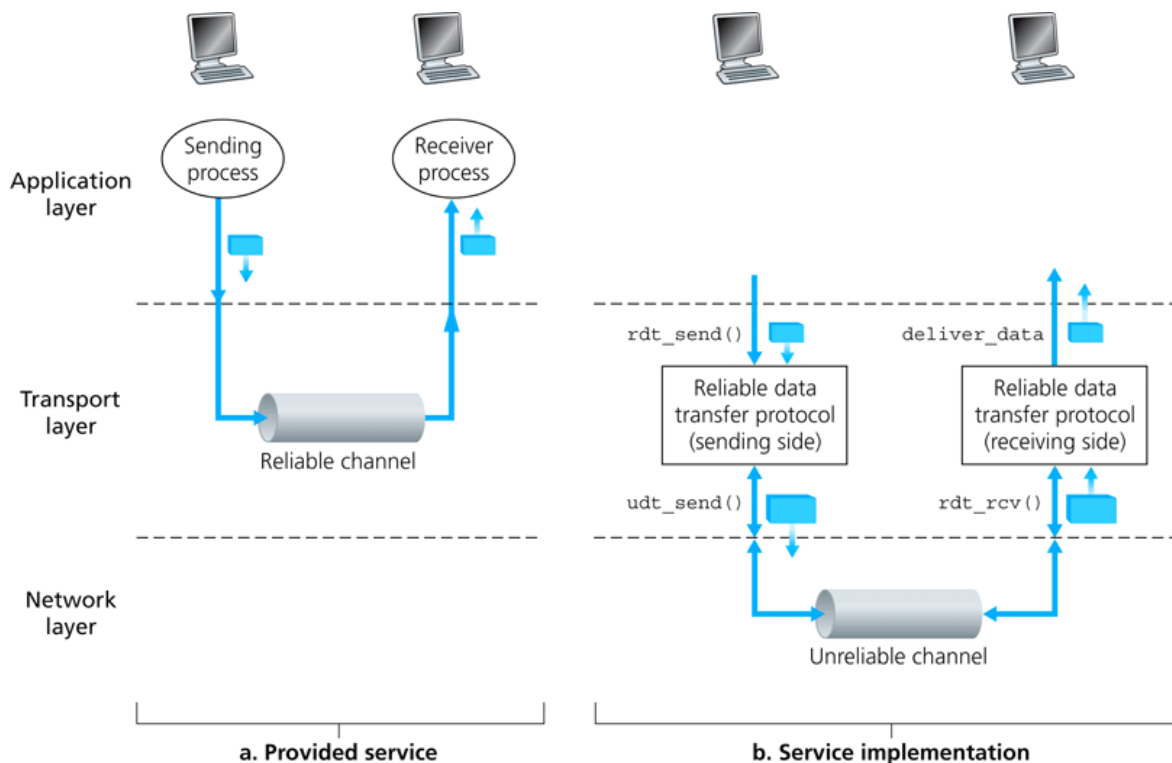
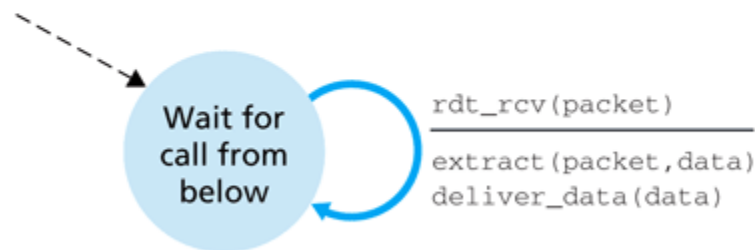


Figure 3.8 ♦ Reliable data transfer: Service model and service implementation

The finite state machines of the sender and receiver are given below:



a. rdt1.0: sending side



b. rdt1.0: receiving side

Figure 3.9 ♦ rdt1.0 – A protocol for a completely reliable channel

The following are the basic implementation steps:

1. Pick a transfer file - BMP image file (recommended), easier identifying loss of packets in an image file (lost pixels).
2. **Make_Packet** function - parse the image file and break it down to several packets (set a fixed packet size, say 1024B).
3. Use the UDP sockets (Phase 1) to send and receive RDT1.0 packets - ONE PACKET AT A TIME.
4. The RDT1.0 receiver should assemble packets in order and deliver the entire transfer file to the application.

Implementation: Individual effort

Expectations: In this phase of the project, each student is expected to gain a thorough understanding of socket programming. In this phase of the project, you will learn about the basic principles used to provide non-pipelined reliable data transfer over a perfectly reliable channel.

Programming language: Python.

Deliverables:

1. **DESIGN_DOCUMENT.pdf/md**
2. **README.pdf/md**
3. **Invite to your PRIVATE GitHub Repo**
4. **GitHub Link**

Submit all your documents in a single compressed file with the name **LastName_Phase1.zip**.

All submissions will be inspected using the [Plagiarism detection software](#).

References:

1. Socket Programming
 - o [JAVA Socket Programming](#)
 - o [Linux Gazette's Socket Programming](#) and [Beej's Guide](#)
 - o [Socket Programming by JAVA World](#) and [O'REILLY JAVA Network Programming](#)
2. UDP: [RFC768](#)
3. Principles of Reliable Data Transfer, Section 3.4.1, Kurose/Ross (7th Ed).